

Звіт

до Лабораторної роботи №10

Узагальнені контейнери та ітератори в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок реалізації власного узагальненого контейнера та власного ітератора в Java.

Також метою роботи є навчитися відокремлювати внутрішню структуру зберігання даних від механізму обходу елементів та забезпечити підтримку циклу for-each через інтерфейс `Iterable<T>`.

2. Варіант роботи

Варіант 5 — ігровий інвентар

Для обраного варіанта було реалізовано просту модель ігрового інвентаря. Основними сутностями є гравець, ігровий предмет і слот спорядження. Для зберігання предметів використовується власний generic-контейнер `InventoryContainer<T>`.

3. Опис предметної області

У лабораторній роботі розглядається предметна область ігрового інвентаря. У такій моделі є гравець, ігрові предмети та слоти спорядження.

Гравець може мати інвентар, у якому зберігаються різні предмети: зброя, броня, кільця, зілля та інші об'єкти.

Для зберігання предметів було реалізовано власний узагальнений контейнер `InventoryContainer<T>`. Він може працювати не тільки з предметами типу `Item`, а й з іншими типами даних, наприклад `String`.

4. Класи предметної області

4.1 Клас Player

Клас `Player` описує гравця.

Поля класу:

- `nickname` — нікнейм гравця;
- `level` — рівень гравця.

Клас містить конструктор, гетери та метод `toString()`. У конструкторі передбачено перевірки: нікнейм не може бути порожнім, а рівень гравця має бути додатним числом.

4.2 Клас Item

Клас `Item` описує ігровий предмет.

Поля класу:

- `name` — назва предмета;
- `slot` — слот спорядження;
- `power` — сила предмета.

Клас містить конструктор, гетери та метод `toString()`. У конструкторі передбачено перевірки: назва предмета не може бути порожньою, слот предмета не може бути `null`, а сила предмета не може бути від'ємною.

4.3 Перелік EquipmentSlot

`EquipmentSlot` реалізовано як `enum`. Він описує допустимі слоти спорядження.

`WEAPON`

`ARMOR`

HELMET

RING

POTION

Використання `enum` є зручним, тому що слот предмета не зберігається довільним рядком. Це зменшує кількість можливих помилок у програмі.

5. Реалізований generic-контейнер

У роботі реалізовано власний generic-контейнер:

```
InventoryContainer<T>
```

Контейнер є узагальненим, тому може зберігати об'єкти різних типів.

Приклади використання:

```
InventoryContainer<Item> inventory = new InventoryContainer<>();
```

```
InventoryContainer<String> notes = new InventoryContainer<>();
```

Це показує, що один і той самий клас контейнера можна повторно використовувати з різними типами без дублювання коду.

Контейнер не є обгорткою над `ArrayList` або іншою стандартною колекцією Java. Він реалізований самостійно на основі внутрішнього масиву.

6. Внутрішнє подання контейнера

Внутрішнє сховище контейнера реалізовано через масив:

```
private Object[] items;
```

Для зберігання кількості реально доданих елементів використовується поле:

```
private int size;
```

Масив `items` відповідає за фізичне зберігання елементів, а `size` показує, скільки елементів зараз знаходиться в контейнері.

Таке рішення дозволяє самостійно реалізувати логіку додавання, видалення, доступу за індексом та обходу елементів.

7. Основні методи контейнера

Метод	Призначення
add(T item)	Додає новий елемент у кінець контейнера.
get(int index)	Повертає елемент за індексом.
remove(int index)	Видаляє елемент за індексом і повертає видалений елемент.
size()	Повертає поточну кількість елементів у контейнері.
capacity()	Повертає поточну місткість внутрішнього масиву.
isEmpty()	Перевіряє, чи є контейнер порожнім.
clear()	Очищує контейнер.
iterator()	Повертає власний ітератор контейнера.

7.1 Метод add(T item)

Метод додає новий елемент у кінець контейнера. Перед додаванням перевіряється, чи не дорівнює елемент null. Якщо місця у внутрішньому масиві вже немає, викликається метод grow(), який збільшує місткість масиву.

7.2 Метод `get(int index)`

Метод повертає елемент за індексом. Перед отриманням елемента виконується перевірка індексу. Якщо індекс неправильний, генерується `IndexOutOfBoundsException`.

7.3 Метод `remove(int index)`

Метод видаляє елемент за індексом і повертає видалений елемент. Після видалення всі наступні елементи зсуваються вліво. Остання зайва позиція масиву очищується, а значення `size` зменшується на одиницю.

8. Автоматичне розширення масиву

У контейнері реалізовано автоматичне розширення внутрішнього масиву. Якщо під час додавання нового елемента масив уже заповнений, викликається приватний метод:

`grow()`

У цьому методі створюється новий масив, який у 2 рази більший за попередній. Після цього всі старі елементи копіюються у новий масив.

Це дозволяє додавати більше елементів, ніж було передбачено початковою місткістю контейнера.

9. Реалізація власного ітератора

Власний ітератор реалізовано як внутрішній клас:

```
private class InventoryIterator implements Iterator<T>
```

Ітератор має поле:

```
private int currentIndex;
```

Це поле зберігає індекс поточного елемента під час обходу контейнера.

Ітератор реалізує два основні методи:

`hasNext()`

`next()`

Метод `hasNext()` перевіряє, чи є ще елементи для обходу. Метод `next()` повертає наступний елемент контейнера. Якщо елементів більше немає, метод `next()` генерує виняток `NoSuchElementException`.

10. Підтримка Iterable<T> і циклу for-each

Клас InventoryContainer<T> реалізує інтерфейс:

Iterable<T>

Для цього в контейнері перевизначено метод:

@Override

public Iterator<T> iterator()

Цей метод повертає новий об'єкт власного ітератора.

Завдяки цьому контейнер можна використовувати у циклі for-each:

```
for (Item item : inventory) {  
    System.out.println(item);  
}
```

Тобто клієнтський код може обходити елементи контейнера, не знаючи, як саме вони зберігаються всередині.

11. UML-діаграма

Для моделі було створено UML-діаграму у файлі docs/uml.puml. На діаграмі показано класи предметної області, generic-контейнер, власний ітератор і демонстраційний клас.

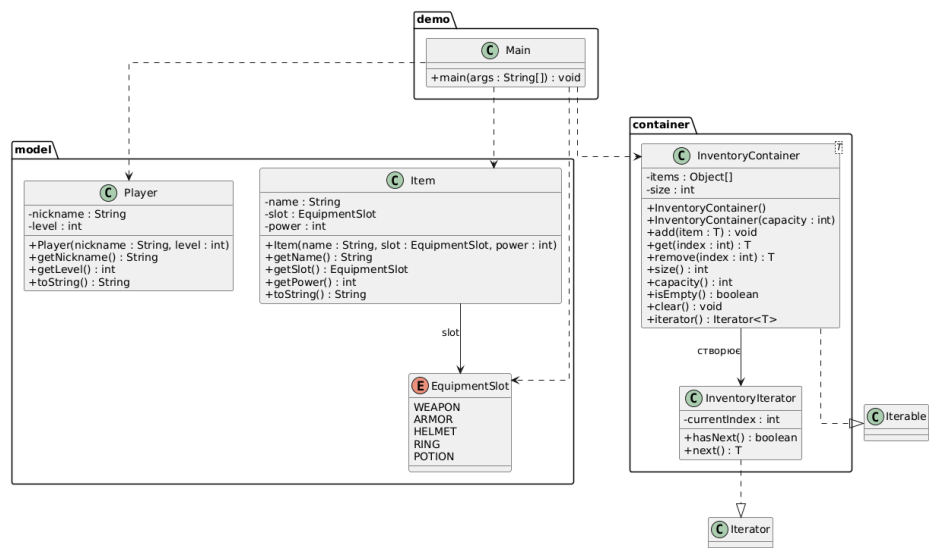


Рисунок 1 — UML-діаграма моделі ігрового інвентаря та generic-контейнера

12. Демонстрація роботи програми

У класі Main демонструється робота ігрового інвентаря.

Послідовність дій у програмі:

1. Створюється об'єкт Player.
2. Створюються кілька об'єктів Item.
3. Створюється контейнер InventoryContainer<Item>.
4. Предмети додаються в контейнер.
5. Виводиться кількість предметів.
6. Виводиться місткість контейнера.
7. Демонструється отримання предмета за індексом.
8. Демонструється обхід через звичайний Iterator.
9. Демонструється обхід через цикл for-each.
10. Видаляється предмет за індексом.
11. Повторно виводиться інвентар після видалення.
12. Додатково демонструється робота generic-контейнера з типом String.

ІНВЕНТАР

Гравець: Artem, рівень: 15

Додавання предметів:

Кількість предметів: 4

Місткість контейнера: 6

Отримання предмета за індексом:

Предмет: Сталевий меч, слот: WEAPON, сила: 25

Обхід через звичайний Iterator:

Предмет: Сталевий меч, слот: WEAPON, сила: 25

Предмет: Залізна броня, слот: ARMOR, сила: 18

Предмет: Кільце сили, слот: RING, сила: 7

Предмет: Зілля лікування, слот: POTION, сила: 10

Обхід через for-each:

Предмет: Сталевий меч, слот: WEAPON, сила: 25

Предмет: Залізна броня, слот: ARMOR, сила: 18

Предмет: Кільце сили, слот: RING, сила: 7

Предмет: Зілля лікування, слот: POTION, сила: 10

Видалення предмета з індексом 1:

Видалено: Предмет: Залізна броня, слот: ARMOR, сила: 18

Інвентар після видалення:

Предмет: Сталевий меч, слот: WEAPON, сила: 25

Предмет: Кільце сили, слот: RING, сила: 7

Предмет: Зілля лікування, слот: POTION, сила: 10

Демонстрація генеріс-контейнера з іншим типом

Ключ від підземелля

Карта локації

Записка NPC

Рисунок 2 і 3 — результати запуску програми

13. Тестування

Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- InventoryContainerTest;
- MainTest.

Тести перевіряють:

- додавання елементів у контейнер;
- отримання елементів за індексом;
- видалення елементів;
- зміну розміру контейнера після додавання та видалення;
- роботу методу isEmpty();
- обхід контейнера через for-each;
- запуск демонстраційної програми Main без помилок.

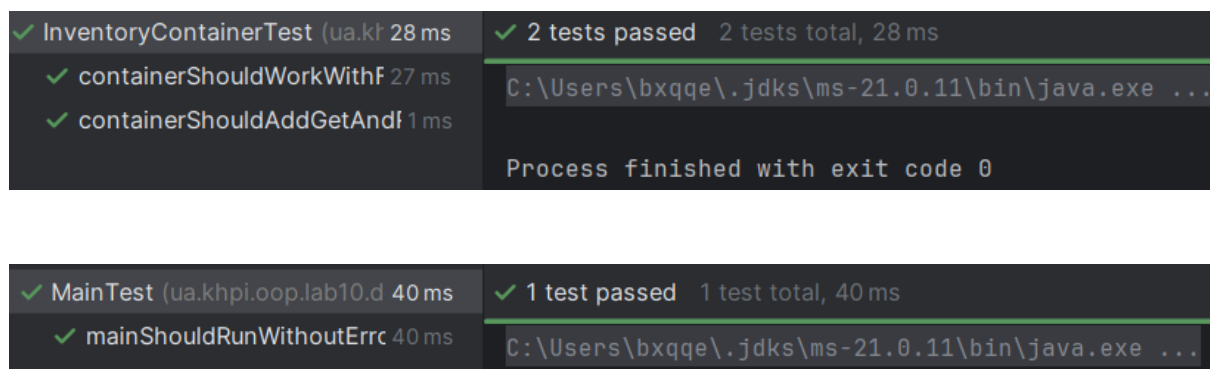


Рисунок 4 і 5 — результати виконання тестів

14. Додаткове завдання

У роботі додатково реалізовано автоматичне розширення внутрішнього масиву.

Якщо кількість елементів досягає поточної місткості, контейнер створює новий масив більшого розміру та копіює в нього старі елементи.

Також реалізовано метод:

clear()

Він дозволяє повністю очистити контейнер. Ці можливості роблять контейнер зручнішим для практичного використання.

15. Висновок

У ході виконання лабораторної роботи було реалізовано модель ігрового інвентаря для варіанта 5.

Було створено класи предметної області: Player, Item і EquipmentSlot.

Для зберігання елементів було реалізовано власний generic-контейнер:

InventoryContainer<T>

Контейнер використовує внутрішній масив Object[], підтримує додавання, доступ за індексом, видалення, визначення розміру, перевірку на порожнечу, очищення та автоматичне розширення.

Також було реалізовано власний ітератор InventoryIterator, який дозволяє послідовно обходити елементи контейнера.

Завдяки реалізації Iterable<T> контейнер можна використовувати у циклі for-each.

Отже, лабораторна робота показала, як працюють самописні generic-контейнери, власні ітератори та механізм підтримки for-each у Java.